Moment 2: Direct Override as the Intervention-Time Exercise of Human Governance in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 1 May 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the second of the two moments at which human governance is exercised in CKS systems — **direct override**, the intervention-time exercise of human authority over substrate content, orchestration rules, and cell-produced state — as a standalone architectural commitment with independent operational content, paired with rule authoring (formalized as Moment 1 in a sibling note) and distinct from the right-axis treatment of override (formalized as the override right in another sibling note).

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "human-governed" commitment names two moments at which governance is exercised: orchestration rule authoring at design time, and direct override at intervention time. The architecture grounds its cost-scaling property — that governance cost does not grow with substrate size — in the fact that neither moment scales with substrate size. A sibling note formalizes Moment 1 (rule authoring); this note formalizes Moment 2 (direct override) as a standalone derivation. The note states what direct override commits to as a moment-axis architectural commitment, expands the cost-property treatment that is the load-bearing claim distinguishing this note from the right-axis treatment of override, distinguishes the moment from three adjacent activities commonly conflated with it (incident response, manual review, rollback), names the failure modes that violate the moment specifically, and provides an operational test. The pair-axis decomposition (Moment 1 plus Moment 2) is what makes the architecture's commitment to where governance is exercised — exhaustively, at exactly two moments, neither of which grows with substrate size — defensible against the misreading that governance must scale with what the substrate carries.

1. Why the direct-override moment needs to be formalized as standalone

The CKS pattern's "human-governed" commitment names two moments at which governance is exercised: orchestration rule authoring at design time, and direct override at intervention time (§3.3 of

the source paper). A separate derivation note treats the joint commitment; another sibling note formalizes Moment 1 as standalone. This note formalizes Moment 2.

The architectural content of direct override is often misread in three directions, each of which collapses an architectural commitment into a practice or workflow. As **incident response**, direct override loses its routine-availability content — interventions become emergency-only, gated on an incident declaration. As **workflow exception handling**, it loses its at-the-time-of-choosing content — interventions become exceptional cases routed through process, not authority moves exercised directly. As **one-shot rollback**, it loses its evolution content — interventions become reversions rather than substrate writes, even when the override is authoring forward, not undoing. Naming the moment is what keeps the commitment visible against these readings, and what makes the cost-scaling property the architecture commits to in §6.3 defensible.

A second motivation is the relationship to the override right, which a sibling note treats as standalone. The override right is one of three rights — alongside inspect and modify — that together constitute the human-governed commitment; direct override as a moment is one of two governance moments at which those rights are exercised. The two formalizations are complementary: the right-axis note establishes that override has independent architectural content from inspect and modify, with the *no-justification-required* property as load-bearing; this note establishes that direct override has independent architectural content from rule authoring, with the *intervention-frequency cost property* as load-bearing. Where the right-axis note expanded the no-justification property in detail, this note treats it briefly and expands the cost-property treatment instead.

2. The direct-override moment, defined precisely

In the CKS pattern, **direct override** is the act by which a human writes — or modifies, deletes, or replaces — substrate content, orchestration rules, or cell-produced state at intervention time, on the human's authority alone, with the change taking effect as substrate state and attributed to the human. The act has six operational components.

(a) Exercised at a moment of the human's choosing. Direct override is intervention-time governance, distinguished from the design-time rule authoring of Moment 1 by when the act occurs relative to cell execution. Rule authoring happens in advance of (or independent of) the cell executions it will govern; direct override happens at intervention points the human chooses, often in response to specific situations the rules did not cover or covered incorrectly.

(b) Bypasses orchestration rules' otherwise-applicable constraints. The override is what lets the human act outside the rules' scope on the specific intervention. Other cell executions continue under the rules; the override applies to the specific substrate content or rule the human acts on.

(c) Takes effect as substrate state on the human's authority alone. Direct override is not architecturally gated on justification, approval, or workflow review; the no-justification property treated in detail in the right-axis sibling note holds for the moment-axis case as well. Deployments may layer logging, audit, or peer review on top; the architectural commit is the human's act.

(d) Itself substrate content with provenance. The override action is recorded — writer, timestamp, what was overridden, and where applicable the rule or default the override bypassed. This is the path-retraceability commitment (§3.1) applied to overrides: the intervention is traceable as substrate state, not held in a parallel audit log.

(e) Reversible by subsequent override. Because override actions are themselves substrate content, they are subject to the same three rights as other substrate content. Later override actions can revert, modify, or layer over the original; the architecture commits to override actions being immediate, not final.

(f) Exercisable in the host environment without specialized intervention runtime. Per the tool-agnosticism commitment (§7.1), direct override is exercisable in commodity tools — the same environments that host substrate content support direct-override writes, through the same human read/write access used for inspection and modification.

The six components together define the direct-override moment as the architecture commits to it. Failing any one fails the moment in the architectural sense, even when other governance components are robustly preserved.

3. The cost properties of direct override as a moment

This is the load-bearing section that distinguishes this note from the right-axis treatment of override.

The architecture's linear-cost commitment (Claim 5; §6.3) names governance cost as not size-proportional. Direct override is one of the two moments where this property holds. The cost analysis has four properties.

Override cost is paid per intervention, not per substrate element. A substrate that grows from a hundred entries to a hundred thousand does not require more direct-override work; what determines override cost is how often humans choose to intervene. The architecture treats this as a distinct cost dimension (intervention frequency) from the substrate-size dimension. A deployment that grows its substrate ten-fold without changing its intervention pattern does not pay ten-fold direct-override cost.

Override cost is paid per intervention, not per cell execution. A cell that runs ten thousand times under stable rules incurs no direct-override cost as long as no human intervenes; cell execution volume and direct-override frequency are independent dimensions. A cell that runs once and is overridden ten times has incurred ten override costs against one execution; the count is on interventions, not on executions.

Override cost grows with intervention frequency, which is bounded by deployment design. A deployment whose rules cover most situations requires few direct-override interventions; a deployment whose rules are sparse, or whose domain is volatile, requires more. The intervention rate is the deployment's design choice — set by how exhaustively rules are authored, how quickly rules are revised, and how the operating context evolves — not a property the substrate forces.

Override cost is paid by humans exercising authority, not by an architectural overhead. There is no per-substrate-element compute cost, no per-rule validation cost, no reconciliation cost imposed by the architecture for override to remain available. The architecture's commitment is to availability, not to computational maintenance. A substrate that has not been overridden in a year carries no accrued debt; a substrate overridden every hour carries the human cost of those hours, no more.

None of these costs scale with substrate size. Naming this is the cost-axis content of why direct override is a moment of governance: intervention happens at points in time independent of how large the substrate has grown. The cost-scaling property of Claim 5 turns on the moment's frequency-bounded character, which the standalone formalization makes explicit.

4. What direct override does NOT require

The standalone treatment of the moment is not a maximalist treatment. Stating precisely what it does not require is what keeps the framing from overstating the source paper.

It does not require justification, approval, or workflow gating as architectural preconditions of effect. The full treatment is in the right-axis sibling note; for moment-axis purposes, what matters is that adding such preconditions converts intervention-frequency cost into per-intervention process cost, which can grow superlinearly in process complexity and break the cost-scaling property.

It does not require predictability or rarity. The architecture does not commit to direct override being scheduled, batched, or occurring at predictable times; interventions happen when humans choose to intervene. The moment is architecturally on-demand and frequency-independent: a deployment that exercises override frequently is exercising it at full architectural scope; one that exercises it rarely is exercising the same moment.

It does not require comprehension of consequence. Whether a direct override is well-considered is outside the architecture's scope. The architectural commitment is that the human can override, with provenance recorded; whether the override is wise is a question about the human and the situation, not about the architecture.

It does not require infrastructure beyond the host's three minimal requirements. Direct override is exercisable on the same host the substrate runs on, through the same human read/write access used by inspection and modification. Specialized intervention infrastructure is not architecturally required.

5. What direct override is NOT

Three adjacent activities are commonly conflated with direct override. Each is a real and reasonable commitment in some other architecture; naming what direct override is not is what prevents the misreading.

Not incident response. Incident response handles unplanned events through escalation chains, on-call rotations, and post-incident review. Direct override in CKS is not emergency-only; it is the routine architectural mechanism for intervention at any moment the human chooses, regardless of whether the intervention responds to an incident. Treating direct override as incident-only converts it into a

process-gated mechanism (override permitted only during a declared incident), which violates the at-the-time-of-choosing component of the moment.

Not manual review. Manual review processes — peer review, editorial review, regulatory review — operate as workflow stages where a designated reviewer evaluates content and approves or rejects it. Direct override is not a review stage; it is a substrate write the reviewer authorizes and commits on their own authority. A deployment can layer manual review on top of direct override; it cannot treat review as a substitute for direct override, because review-as-process is gated and direct override is not.

Not rollback. Rollback reverts a system to a prior state — undoing recent changes, restoring from backup, applying a previous version. Direct override may include rollback-shaped actions, but it is broader: it includes any intervention-time write, including writes with no rollback-shape (authoring new substrate content outside any rule's scope, or modifying a rule mid-operation without reverting to a prior rule state). Treating direct override as rollback-only loses the forward-acting content of the moment.

6. How the two moments compose

The architecture's commitment to where governance is exercised is exhausted by the pair: rule authoring at design time, direct override at intervention time. Three observations close the pair.

The two moments are temporally distinct. Rule authoring happens in advance of or independent from the cell executions the rule will govern; direct override happens at points the human chooses, often in response to situations the rules did not cover. A moment that collapses the two times collapses the cost separation that follows.

The two moments are architecturally complementary. Rule authoring produces the rules under which cells routinely operate; direct override is the recourse when those rules misfire, miss context, or do not yet exist for the situation at hand. Without rule authoring, every cell execution would require direct human attention and the architecture would not scale; without direct override, rule authoring would face an impossible bar (rules must be exhaustive, since there is no escape hatch). Each moment makes the other tractable.

The two moments are cost-independent. Rule-authoring cost grows with rule variety; direct-override cost grows with intervention frequency; neither grows with substrate size or with cell execution volume. A deployment can independently tune the two: tighter rules reduce intervention frequency at higher authoring cost; looser rules reduce authoring cost at higher intervention frequency. The choice is the deployment's; the architecture supports either pattern.

There is no third moment. Governance does not happen during cell execution (that is rule application, under rules already authored), and it does not happen during substrate growth (content accumulates under rules and overrides, neither of which is itself a governance moment for content already in the substrate). Naming the two moments — and only the two — is what makes the architecture's commitment to where governance is exercised both precise and defensible.

7. Failure modes that violate the direct-override moment

A system can fail the direct-override moment specifically, even when other governance components are preserved. Six failure modes name the most common ways this happens.

(a) Override gated on incident declaration. When direct override is permitted only after an incident, error condition, or exceptional situation is declared, the at-the-time-of-choosing component is violated. The architecture does not require a qualifying condition.

(b) Override gated on per-intervention process. When each override action requires a justification, approval, review, or other process step before taking effect, intervention-frequency cost grows with process complexity. The full treatment of why per-intervention process gating is architecturally distinct from process-as-deployment-layer is in the right-axis sibling note; the moment-axis consequence is the cost-model break.

(c) Override scheduled rather than on-demand. When the architecture only permits override during scheduled windows (review meetings, batched intervention sessions, designated update windows), the on-demand content of the moment is lost. The intervention is no longer the human's choice; it is the schedule's.

(d) Override that requires specialized intervention infrastructure. When direct override is exercisable only through dedicated intervention systems separate from the substrate's host environment, the tool-agnosticism content of the moment is violated.

(e) Override actions held outside the substrate. When override actions are recorded only in audit logs, intervention systems, or operational logs separate from the substrate, the path-retraceability content is lost. Override actions are themselves substrate content; their provenance lives in the substrate.

(f) Irreversible override. When the architecture treats override actions as final — not subject to subsequent override by other humans with override authority — the moment's reversibility content is violated. Override actions are immediate, not final; they remain subject to the same three rights as other substrate content.

A system that exhibits any of (a)–(f) does not implement the direct-override moment specifically, and naming the failure precisely is what allows downstream remediation.

8. Operational test

A system implements the direct-override moment specifically if and only if all of the following are true at all times during the substrate's existence:

1. Direct override is exercisable at any time the authorized human chooses, without architectural requirement of incident declaration, scheduled window, or qualifying condition.
2. Override actions take effect as substrate state immediately on the human's authority, with provenance recorded in the substrate (writer, timestamp, what was overridden, and where

applicable the rule or default the override bypassed).

3. Override does not require justification, approval, or workflow gating as architectural preconditions of effect; the no-justification property of the right-axis sibling note holds for the moment-axis case as well.
4. Override is exercisable on the same host environment as the rest of the substrate's operation, through the same human read/write access used for inspection and modification, with no specialized intervention infrastructure as architectural requirement.
5. Override actions are themselves substrate content, subject to the three rights, including subsequent override by other humans with override authority.
6. The architectural cost of override is paid per intervention, growing with intervention frequency only, independent of substrate size and cell execution volume.

A system that fails any of (1)–(6) does not implement the direct-override moment in the architectural sense, even if it supports override-like activity in some other form. Such a system may be useful for other purposes but is not CKS-coherent on this axis of governance.

9. Conclusion

Implementations that conflate direct override with incident response produce systems where intervention is gated on emergency declaration, with consequences for cost (incident processes are themselves expensive) and accessibility (declaring incidents requires authority and judgment that non-specialists may not have; §7.4). Implementations that conflate direct override with manual review produce systems where intervention is a workflow stage rather than a substrate write, failing the direct-access content of the moment. Implementations that conflate override with rollback lose the forward-acting content of the moment, treating intervention as undo rather than as authority.

Naming the moment as standalone — paired with the rule-authoring moment formalized in the sibling Moment 1 note — gives downstream implementers a precise specification of what their intervention mechanism must satisfy, independent of how cell execution and rule authoring are handled. The two moments together exhaust the architecture's commitment to where governance is exercised; the pair-axis decomposition is what makes the cost-scaling property of Claim 5 defensible against the misreading that governance must scale with substrate size.

Subsequent work that implements, extends, or argues against the CKS commitment to direct override should use the term in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Moment 2: Direct Override as the Intervention-Time Exercise of Human Governance in the Coordination Knowledge Substrate Pattern.* 1 May 2026. ORCID: 0009-0004-8065-3235.